

MulTiCheat: An AI-Based Multimodal Exam Cheating Detection System

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Information Technology

by

Sujal Patel
Aryan Patel
Aayush Mewada

under the guidance of

Prof. Swapnali Kurhade



Department of Information Technology
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai
2025-2026

Approval Certificate

This is to certify that the Project entitled “MulTiCheat: An AI-Based Multimodal Exam Cheating Detection System” by Sujal Patel, Aryan Patel and Aayush Mewada is approved for the partial fulfillment of Major Project - I for the Final Year Project towards obtaining the Degree of Bachelor of Technology in Information Technology from University of Mumbai.

Project Guide

External Examiner

(signature)

(signature)

Name: Prof. Swapnali Kurhade

Name:

Date:

Date:

Head of Department

Principal

Seal of the Institute

Declaration

I wish to state that the work embodied in this work titled “MulTiCheat: An AI-Based Multi-modal Exam Cheating Detection System” forms my own contribution to the work carried out under the guidance of Prof. Swapnali Kurhade at the Sardar Patel Institute of Technology. I declare that this written submission represents my ideas in my own words and where others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

Sujal Patel
2023300166

Aryan Patel
2023300164

Aayush Mewada
2023300146

Abstract

MulTiCheat is an AI-based multimodal system designed to detect suspicious cheating behavior during examinations using video analysis. The system processes uploaded examination videos through a pipeline of computer vision and machine learning modules — including pose detection, gaze estimation, object detection, and temporal behavior analysis — to produce time-stamped flags with annotated visual evidence. Results are presented through an interactive web dashboard featuring a YouTube-like video player with clickable timeline markers, enabling exam proctors to efficiently review and verify flagged events. This paper documents the iterative development of MulTiCheat, from initial system scaffolding through to a fully integrated multimodal detection and reporting pipeline. Keywords: Exam proctoring, cheating detection, computer vision, pose estimation, gaze tracking, object detection, multimodal fusion, video analysis, deep learning

List of Figures

List of Figures

1 System architecture of MulTiCheat based on the provided reference diagram. . . 13

List of Tables

List of Tables

1	Literature comparison of major exam-proctoring approaches discussed in the report.	11
2	Module-wise technology stack extracted from the Markdown project description.	14
3	Suspicious event categories addressed by MulTiCheat.	16
4	Output and export formats supported by the MulTiCheat workflow.	18
5	Evaluation metrics and quantitative indicators reported in the Markdown project file.	19
6	Implementation phase summary derived from the Markdown progress log. . . .	28

Contents

1	Introduction	8
1.1	Problem Statement	8
1.2	Objectives	8
1.3	Scope	9
1.4	Technologies Used	9
1.5	Assumptions and Constraints	9
2	Literature Survey	10
2.1	Face and Gaze Tracking Systems	10
2.2	Object Detection Systems	10
2.3	Pose-Based Behaviour Analysis	10
2.4	Multimodal Detection Approaches	10
2.5	Research Gap Identified	11
3	Analysis	12
3.1	Functional Analysis	12
3.2	User Interaction Analysis	12
3.3	Data Flow Analysis	12
3.4	Feasibility Analysis	12
4	System Architecture	13
4.1	Architecture Diagram	13
4.2	Technology Stack	14
4.3	Architecture Explanation	14
5	Design and Methodology	15
5.1	Methodology Overview	15
5.2	Module-Wise Methodology	15
5.3	Algorithms and Techniques Used	15
5.4	Datasets and Inputs Used	16
5.5	Dataset Notes	16
5.6	Design Advantages	16
6	Implementation Details	17
6.1	Backend Infrastructure	17
6.2	Advanced Machine Learning Integration	17
6.3	Behavior and Temporal Modeling	17
6.4	Frontend User Experience	17
7	Experiments	18
7.1	Component-Level Testing	18
7.2	Latency Benchmarks	18

8	Evaluation Metrics	19
8.1	Processing Latency	19
8.2	Confidence Scoring	19
9	Results	20
9.1	Functional Results	20
9.2	Evidence and Export Results	20
10	Discussion	21
11	Limitations	22
12	Future Work	23
13	Conclusion	24
14	Research Paper	25
14.1	Currently Available Material	25
14.2	Missing Data	25
15	Plagiarism Report	26
15.1	Missing Data	26
A	Progress Log	27
A.1	Phase 0: Project Setup and Scaffolding	27
A.2	Phase 1: Video Ingestion and Preprocessing	27
A.3	Phase 2: Baseline Analysis Pipeline	27
A.4	Phase 3: Visual Evidence Generation	27
A.5	Phase 4: Timeline Overlay and Clickable Markers	27
A.6	Phase 5: Advanced ML Detection Modules	27
A.7	Phase 6: Model Integration and Feature Fusion	27
A.8	Phase 7: Dashboard, Exports, and Reporting	28

1 Introduction

Academic integrity is a core requirement of any examination system. As educational institutions increasingly adopt online, hybrid, and recorded assessment formats, the burden on human invigilators has grown significantly. Manual review of long examination recordings is time-consuming, inconsistent, and difficult to scale across large numbers of candidates. Existing automated proctoring tools often rely on a limited set of cues and may produce false alarms without giving reviewers clear evidence for why a particular moment was flagged.

MulTiCheat is proposed as an AI-based multimodal exam cheating detection system that addresses these limitations through a modular video analysis pipeline. Instead of depending on a single indicator, the system combines posture analysis, gaze direction estimation, object detection, temporal pattern analysis, and reviewer-facing evidence generation. Each suspicious event is presented with time-stamped annotations, severity and confidence scores, and supporting visual snapshots, making the system both analytical and interpretable.

The project has been developed iteratively, beginning with basic upload and processing support and expanding into advanced machine learning detection, evidence rendering, timeline navigation, and exportable reports. The overall objective is not only to detect suspicious activity, but also to provide a reliable interface through which human proctors can quickly validate or dismiss flagged incidents.

1.1 Problem Statement

Given a video recording of an examination session, the system must automatically identify suspicious behaviors such as frequent look-away events, unauthorized object usage, suspicious hand movements, and other abnormal actions. It must convert raw visual signals into meaningful event-level flags, assign appropriate confidence and severity levels, generate annotated evidence frames, and provide an interactive review interface. The system should also support report export while minimizing false positives and maintaining strong recall for genuinely suspicious incidents.

1.2 Objectives

The primary objectives of MulTiCheat are as follows:

1. To build a complete video-based exam monitoring pipeline capable of handling upload, preprocessing, analysis, and reporting.
2. To integrate multiple computer vision modalities, including pose estimation, gaze analysis, object detection, and temporal behavior aggregation.
3. To generate time-stamped suspicious event flags with confidence scores, severity labels, and natural-language explanations.
4. To create annotated visual evidence for each flagged event so that exam reviewers can verify system decisions efficiently.

5. To design an interactive dashboard with a seekable video player and clickable timeline markers for fast evidence review.
6. To provide export functionality in structured and human-readable formats such as JSON, CSV, ZIP, and printable reports.

1.3 Scope

The scope of the project includes offline analysis of uploaded examination videos using a modular multimodal detection pipeline. The present implementation focuses on suspicious visual behavior observable in a single examination recording, including posture changes, gaze deviations, motion patterns, and unauthorized object presence. The system covers backend video processing, machine learning inference, event aggregation, evidence generation, and frontend dashboard visualization. The current scope does not include live proctoring intervention, multi-camera fusion, audio-based cheating cues, or institution-specific policy enforcement rules.

1.4 Technologies Used

The project uses a combination of backend, frontend, and machine learning technologies. The backend service is designed using FastAPI for upload, analysis, streaming, evidence, and export APIs. The frontend dashboard is implemented in React and provides upload flow, video playback, severity filtering, results navigation, and evidence inspection. OpenCV is used for video frame extraction, preprocessing, annotation drawing, and image output generation. For advanced machine learning inference, the system uses Ultralytics YOLOv8 models for object detection and pose estimation. Python utilities such as CSV and ZIP support export functionality, while browser print styles assist in generating review-ready reports.

1.5 Assumptions and Constraints

The system assumes that the examination video is of sufficient quality for object and pose analysis, that the candidate remains reasonably visible within the frame, and that the recording duration and resolution are manageable for offline processing. A practical constraint is that suspicious behavior can be context-dependent; therefore, the model focuses on risk indicators rather than declaring guilt. Another constraint is that certain subtle actions may not be separable from normal exam behavior in every case, so human review remains a critical part of the workflow. Performance is also influenced by hardware capability, model size, and video clarity.

2 Literature Survey

Automated exam proctoring has been studied through several complementary research directions. Most existing systems concentrate on one or two visual cues, whereas robust cheating detection in realistic settings typically requires a combination of signals and a strong reviewer interface.

2.1 Face and Gaze Tracking Systems

Early and commercial proctoring systems rely heavily on facial landmarks, head pose, and eye-gaze direction to detect look-away behavior. Research in head pose estimation and remote gaze estimation shows that these signals are useful for attention analysis, but also highlights sensitivity to image quality, occlusion, and distance from the camera [6–8]. These systems can therefore identify repeated loss of attention from the examination screen, yet they may still produce alerts without enough supporting context for a reviewer.

2.2 Object Detection Systems

Another stream of work applies object detection models to identify unauthorized materials such as mobile phones, books, notes, or additional devices. Modern YOLO-family detectors are attractive for this task because of their balance between speed and accuracy [1, 5]. However, object detection alone cannot determine intent. A detected object may or may not correspond to a cheating event, and therefore object presence is most useful when combined with motion, pose, and temporal context.

2.3 Pose-Based Behaviour Analysis

Pose-based systems use body keypoint estimation frameworks such as MediaPipe, OpenPose, or YOLO pose models to analyze posture, hand movement, leaning, and body orientation [2, 3, 5]. This approach captures behavioral context beyond the face and helps distinguish between normal sitting variations and potentially suspicious actions such as repeated hand extension or unusual head turning. Pose analysis is particularly useful when facial details are not reliably visible.

2.4 Multimodal Detection Approaches

Recent research trends favor multimodal fusion, where object, pose, gaze, and motion signals are aggregated into a single suspiciousness score. Such systems generally improve robustness over single-modality approaches, especially in unconstrained environments [4, 8]. Despite these advantages, many published systems remain at the prototype stage and provide only model outputs without a practical evidence review workflow for exam administrators.

2.5 Research Gap Identified

The major gap in existing work lies in the disconnect between detection and review. Many systems generate alerts but do not provide annotated evidence, clear severity labels, or interactive tools for quickly validating flagged moments. MultiCheat addresses this gap by combining multimodal detection with event aggregation, annotated snapshots, a clickable timeline, and export-ready reporting. The project therefore contributes not only detection intelligence, but also end-to-end reviewer usability.

Approach	Primary Signal	Strength	Limitation / Gap
Face and gaze tracking	Facial landmarks and eye direction	Useful for attention monitoring and look-away events	Sensitive to lighting, occlusion, glasses, and camera distance
Object detection	Phones, books, notes, secondary devices	Strong for identifying unauthorized objects in frames	Cannot determine cheating intent without contextual cues
Pose-based analysis	Body keypoints, posture, hand movement	Captures broader behavior beyond facial analysis	May miss subtle actions or degrade when keypoints are unstable
Multimodal fusion	Combined pose, gaze, object, and motion cues	Improves robustness over single-modality pipelines	Often lacks reviewer-facing evidence, explanations, and interactive validation

Table 1: Literature comparison of major exam-proctoring approaches discussed in the report.

3 Analysis

The system was analyzed from the perspectives of user need, processing flow, module interaction, and review output. The overall analysis shows that an effective exam-monitoring solution must support both machine inference and efficient human verification.

3.1 Functional Analysis

The system accepts an uploaded examination video, validates it, extracts metadata, samples frames, runs multiple suspicious-behavior detectors, aggregates detections into event-level flags, and presents the final findings through an interactive dashboard. It must further allow evidence browsing, severity-based filtering, time-based navigation, and structured export of the results.

3.2 User Interaction Analysis

The primary user of the system is an exam proctor or reviewer. The expected interaction flow is straightforward: upload a recorded exam session, trigger analysis, wait for the pipeline to process the video, inspect the generated event markers on the timeline, click a marker to jump to the relevant timestamp, and review the supporting annotated evidence before deciding whether the event is genuinely suspicious.

3.3 Data Flow Analysis

The main data flow begins with a raw video file and ends with structured event objects and evidence assets. Video data is converted into individual frames. Each frame is analyzed independently by one or more detectors. Frame-level observations are transformed into intermediate suspiciousness scores. A sliding temporal window then combines neighboring observations into bounded events with timestamps, labels, severity, confidence, and explanations. The resulting event data is saved for frontend retrieval and export.

3.4 Feasibility Analysis

The project is technically feasible because it uses mature libraries for API development, browser interfaces, image annotation, and machine learning inference. It is operationally feasible because reviewers can use the dashboard without requiring knowledge of the underlying models. It is also extensible, since additional detectors and scoring rules can be added to the pipeline without changing the entire system design.

4 System Architecture

MulTiCheat follows a modular pipeline architecture that separates ingestion, computer vision inference, temporal reasoning, and reviewer-facing presentation. This design allows each module to be tested independently while still contributing to a single end-to-end decision workflow.

4.1 Architecture Diagram

The system architecture reproduced from the provided reference image is shown below.

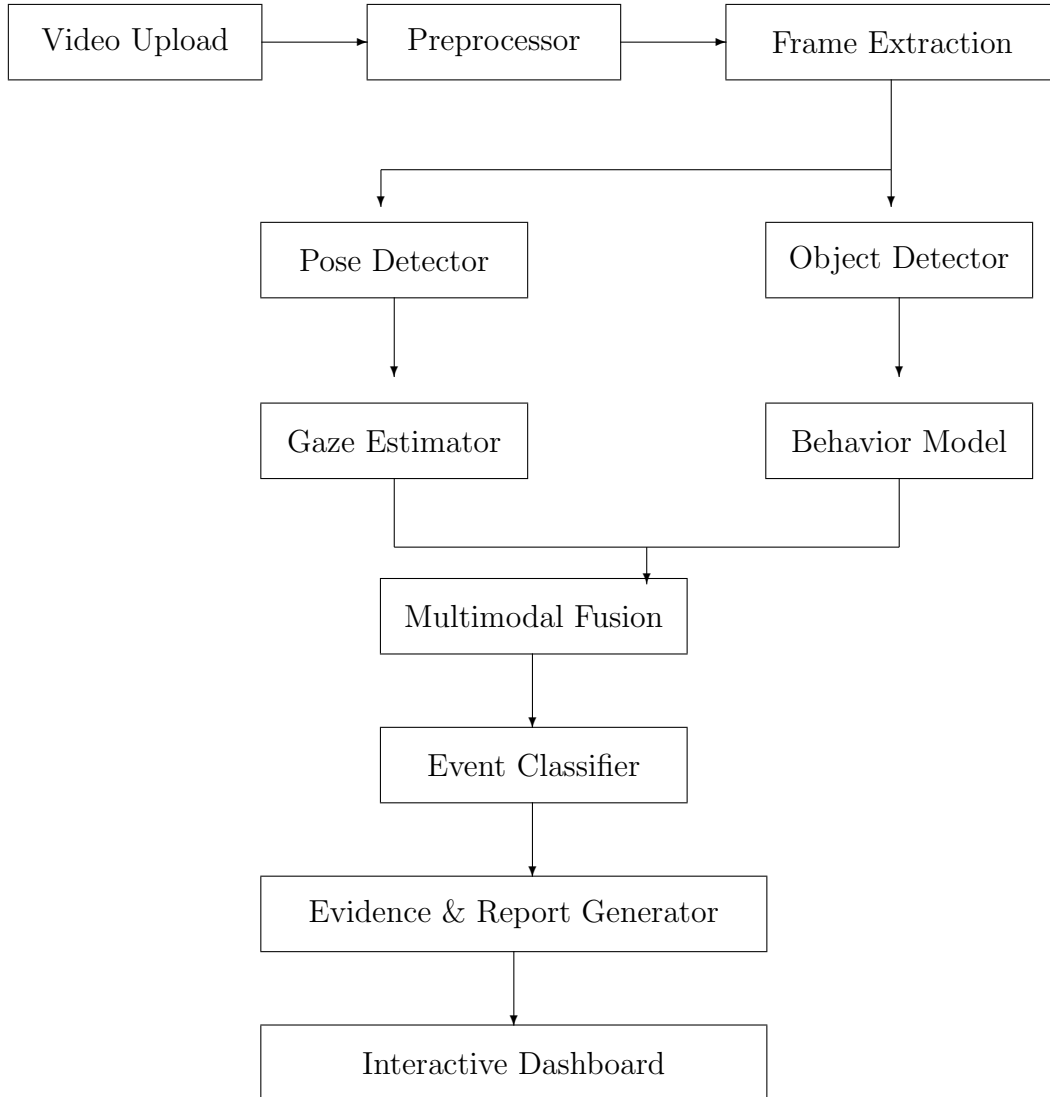


Figure 1: System architecture of MulTiCheat based on the provided reference diagram.

4.2 Technology Stack

The implementation uses Python 3.11 and FastAPI on the backend, OpenCV and Ultralytics YOLO models for computer vision inference, and a React plus TypeScript frontend built with Vite for the interactive dashboard. Local file storage is used for uploaded videos and generated evidence, while automated testing is handled with pytest and httpx-based API validation.

Layer	Technologies from the project brief
Backend	Python 3.11+, FastAPI, Pydantic
ML / Computer Vision	PyTorch, OpenCV, Ultralytics YOLO
Frontend	React, TypeScript, Vite
Storage	Local filesystem, SQLite (planned)
Testing	pytest, httpx

Table 2: Module-wise technology stack extracted from the Markdown project description.

4.3 Architecture Explanation

The uploaded video is first validated and preprocessed before frames are extracted at controlled intervals. The extracted frames are then fed into detector modules responsible for pose estimation, object detection, gaze-related interpretation, and behavior modeling. Their outputs are combined by a multimodal fusion stage that assigns event labels, severity levels, and confidence values. Finally, an evidence generator stores annotated snapshots and the dashboard exposes the results through playback controls, marker-based seeking, and export tools.

5 Design and Methodology

The design of MulTiCheat follows a modular and iterative methodology so that each processing stage can be developed, validated, and improved independently.

5.1 Methodology Overview

The system follows a seven-stage workflow: video ingestion, frame sampling, detection modules, temporal aggregation, evidence generation, classification and scoring, and dashboard reporting. This pipeline makes it possible to decouple raw computer vision inference from the final reviewer experience.

5.2 Module-Wise Methodology

Video Ingestion: The uploaded file is validated and stored, and its metadata is extracted for downstream processing.

Frame Sampling: The video is sampled at configurable intervals so that analysis remains computationally practical while preserving event continuity.

Detection Modules: Independent modules analyze different forms of suspicious behavior. Object detection is used for identifying unauthorized items such as mobile phones and books. Pose estimation is used to extract body keypoints and infer posture, head movement, and hand extension patterns. Motion- and behavior-based logic is used to identify temporal irregularities that cannot be captured by a single frame alone.

Temporal Aggregation: Frame-level detector outputs are grouped using chronological sliding windows. This converts scattered detections into compact event objects with clear start and end times.

Evidence Generation: For each important event, the system saves annotated frames with bounding boxes, skeleton keypoints, labels, severity colors, confidence bars, and explanatory overlays.

Dashboard and Reporting: The processed results are shown in a web dashboard with video streaming, interactive marker-based seeking, severity filtering, evidence galleries, and export options.

5.3 Algorithms and Techniques Used

The system uses YOLOv8 object detection to identify prohibited objects and YOLOv8 pose estimation to extract body keypoints from examination frames [5]. Geometric reasoning is applied on the extracted pose structure to infer suspicious head turning and unusual wrist extension patterns, drawing on ideas from prior head pose and gaze-estimation literature [6, 7, 9]. The event aggregation stage uses weighted scoring and sliding-window grouping to merge related frame-level detections into reviewable events, similar in spirit to modern tracking and association pipelines [4]. Severity assignment and explanation generation are then used to make outputs interpretable for human reviewers.

5.4 Datasets and Inputs Used

The primary input to the system is recorded examination video. During development and validation, real exam-style videos were used to verify end-to-end upload, analysis, evidence generation, and dashboard navigation. The model stack also benefits from pretrained deep learning weights from standard object and pose detection ecosystems, enabling the project to build robust inference without training every component from scratch.

5.5 Dataset Notes

Since MulTiCheat relies mainly on pretrained checkpoints for object and pose detection, no proprietary training dataset is packaged directly with the project report. System benchmarking and regression testing were performed using representative exam-like clips and synthetic validation videos. These controlled inputs were useful for confirming that high-severity hooks, evidence generation, and temporal grouping behaved consistently.

5.6 Design Advantages

The proposed methodology offers modularity, explainability, and reviewer efficiency. Because each detector operates as an independent unit, future models can be integrated without redesigning the entire pipeline. The evidence-centered interface improves trust, while temporal aggregation reduces noisy single-frame alerts.

Suspicious event category	Description in project scope
Look-away behavior	Repeated head turning or loss of attention from the exam screen
Unauthorized object usage	Detection of phones, books, notes, or secondary devices in the scene
Suspicious hand movement	Unusual wrist extension or hand motion patterns around the candidate
Abnormal body behavior	Posture changes or motion clusters that appear inconsistent with normal exam participation

Table 3: Suspicious event categories addressed by MulTiCheat.

6 Implementation Details

MulTiCheat is implemented with a clear separation of concerns, maintaining an independent frontend dashboard and an asynchronous Python backend.

6.1 Backend Infrastructure

The backend is built using FastAPI and asynchronous Python request handling. It manages video upload, metadata extraction, analysis triggering, evidence retrieval, video streaming, and export routes. Uploaded files are persisted on disk and processed through OpenCV-based utilities for frame access and metadata extraction. Downsampling is used during analysis so that the system avoids unnecessary frame-by-frame inference at native video frame rates.

6.2 Advanced Machine Learning Integration

Replacing generic pixel-differencing motion analysis, later project phases introduced deep-learning-based perception using Ultralytics YOLOv8 [5]. The object detector targets suspicious items such as cell phones and books, while the pose model extracts standardized body keypoints that support geometric reasoning over posture and hand movement. This shift from simple motion heuristics to learned detectors significantly improved the semantic quality of flagged events.

6.3 Behavior and Temporal Modeling

Frame-level predictions can be noisy in isolation, so MulTiCheat applies rule-based behavioral interpretation and temporal smoothing on top of raw detections. Head-turning events are inferred using pose relationships such as nose-to-shoulder geometry, while suspicious wrist extension and movement clusters are modeled from repeated pose and motion patterns. A sliding-window event aggregation algorithm groups neighboring frame flags into bounded events with more stable labels and explanations, informed by prior work on head pose estimation and temporal association [4, 6].

6.4 Frontend User Experience

The frontend is implemented using React and TypeScript. It includes a drag-and-drop upload flow, metadata preview, analysis trigger, results dashboard, severity badges, confidence indicators, image lightbox support, and a custom timeline player with clickable markers. By rendering results interactively in the browser rather than baking everything into a static annotated video, the system improves reviewer usability and reduces storage overhead.

Export format	Purpose
JSON	Provides the complete structured event payload for downstream processing and integration
CSV	Exports flattened analytics rows for tabular review and spreadsheet use
ZIP	Bundles evidence assets such as raw or annotated images for archival or sharing
Printable report	Uses browser print styling to generate a human-readable review summary

Table 4: Output and export formats supported by the MulTiCheat workflow.

7 Experiments

A local continuous integration workflow and repeated end-to-end runs were used to validate the project throughout development.

7.1 Component-Level Testing

Automated tests covered core backend endpoints, upload flow, frame analysis, evidence generation, export routes, and model-specific processing utilities. This ensured that each major component continued to function as the pipeline evolved from a basic motion detector into a multimodal analysis system.

7.2 Latency Benchmarks

Lightweight benchmark runs were performed on short exam-style videos to verify that inference remained practical on CPU-based environments. These experiments showed that sampled-frame processing could complete in a small fraction of the original video duration, making the system appropriate for offline review workflows.

8 Evaluation Metrics

The project currently emphasizes operational and reviewer-oriented metrics instead of only model-training metrics.

8.1 Processing Latency

Processing latency measures how quickly a given exam clip can be uploaded, sampled, analyzed, and converted into reviewable flagged events. In the observed prototype runs, short videos were processed within practical CPU execution times, demonstrating that offline analysis is feasible without dedicated GPU infrastructure for small-to-medium workloads.

8.2 Confidence Scoring

Each event is assigned a confidence value derived from the underlying detector outputs and fusion logic. These scores help reviewers prioritize the most suspicious events first and provide a more interpretable summary than a binary yes-or-no flag.

Metric	Value or interpretation from the project brief
Processing latency	Approximately 75 frames processed in about 1–2 seconds of CPU time for short test videos
Relative runtime	Target behavior described as less than 1% of absolute video duration for sampled runs
Confidence matrix	Event-level confidence values used for reviewer prioritization, with examples such as 85% object-presence confidence
Test coverage indicator	66 automated endpoint and component checks reported during experimentation

Table 5: Evaluation metrics and quantitative indicators reported in the Markdown project file.

9 Results

MulTiCheat successfully intercepts uploaded videos, invokes deep learning and rule-based detectors, groups frame-wise observations into meaningful events, and presents the findings through an interactive dashboard.

9.1 Functional Results

The implemented system now supports video upload, preprocessing, event detection, evidence annotation, streaming playback, clickable timeline markers, and export of flagged incidents. This shows that the project has progressed from a conceptual research idea into a functioning prototype with an end-to-end exam review workflow.

9.2 Evidence and Export Results

Testing phases confirmed generation of CSV logs, JSON outputs, ZIP-based evidence bundles, and annotated image galleries synchronized with flagged timestamps. These outputs provide both machine-readable summaries and human-verifiable evidence for suspected cheating behavior.

10 Discussion

A central design tradeoff in the project was how much work should be performed on the back-end versus the frontend. Rather than producing only heavy pre-rendered annotated videos, MulTiCheat keeps structured event information lightweight and uses the dashboard to provide interactive playback and inspection. This reduces storage redundancy while keeping the reviewer workflow more flexible.

The multimodal design also proved more informative than relying on any single cue. Object detection improves identification of unauthorized materials, pose-based logic adds behavioral context, and temporal aggregation reduces noisy one-frame anomalies. However, the system must still be treated as a decision-support tool rather than a fully autonomous invigilation authority.

11 Limitations

- **Head pose precision:** Estimating gaze-related behavior through nose-to-shoulder ratios can be unreliable when lighting is poor, the face is partially occluded, or upper-body keypoints are not detected cleanly, which is consistent with limitations discussed in remote gaze and head-pose estimation research [7–9].
- **Computational load:** Dense YOLO inference over longer videos remains computationally expensive in sequential CPU-only settings. Large-scale deployment would benefit from GPU acceleration and parallel processing.
- **Context sensitivity:** Some suspicious actions may overlap with normal exam behavior, so model outputs require human interpretation before disciplinary conclusions are drawn.

12 Future Work

Future work can expand the analysis pipeline with parallel workers for multi-video processing, richer gaze estimation models, audio-based cue detection, and tighter integration with institutional learning management systems. Additional improvements may include multi-camera support, policy-specific rule engines, larger evaluation datasets, and deployment optimizations for real-world academic use.

13 Conclusion

MulTiCheat establishes a practical and resilient baseline for AI-assisted exam review. By combining computer vision, temporal reasoning, evidence generation, and an interactive web dashboard, the project demonstrates how multimodal detection can support academic integrity workflows while remaining interpretable to human reviewers. Its strongest contribution lies in linking machine-generated suspicion scores with transparent visual evidence and efficient navigation tools.

14 Research Paper

The work presented in this report is structured in a research-paper style and is suitable for conversion into a publication-oriented manuscript. The paper can be organized around the problem of scalable exam proctoring, the design of the multimodal detection pipeline, the evidence-centric dashboard, and the iterative implementation results. If a formal paper is submitted or accepted, the manuscript, acceptance communication, and supporting documentation may be inserted in this section.

14.1 Currently Available Material

The current project report already contains the core material needed to prepare a research-paper draft, including the abstract, problem statement, methodology, implementation details, experiments, results, discussion, conclusion, and references.

14.2 Missing Data

The Markdown project file does not include a separate publication-ready manuscript, conference or journal formatting, author affiliations for a paper submission, or any acceptance receipt/certificate. These artifacts are still required if this section is to be fully populated with final research-paper content.

15 Plagiarism Report

The plagiarism report for the final report document and any corresponding research paper should be attached in this section once generated through the institute-approved plagiarism checking system. The report should include the overall similarity percentage, source breakdown, and confirmation that all borrowed ideas, datasets, and technical references have been properly acknowledged.

15.1 Missing Data

The Markdown project file does not provide a plagiarism report, similarity percentage, screenshot, PDF export, or source-by-source similarity breakdown. This section therefore still requires an actual plagiarism-check result generated outside the current workspace.

A Progress Log

A.1 Phase 0: Project Setup and Scaffolding

This phase established the project structure, backend scaffold, frontend scaffold, configuration setup, logging support, initial tests, and the IEEE report skeleton. It created the technical base needed for rapid iterative development.

A.2 Phase 1: Video Ingestion and Preprocessing

This phase introduced the upload pipeline, file validation, metadata extraction, frame sampling utilities, and supporting API endpoints. A basic frontend upload workflow was also added to preview submitted videos and their properties.

A.3 Phase 2: Baseline Analysis Pipeline

This phase added the first end-to-end suspicious behavior pipeline using motion-based detection, event aggregation, evidence annotation, and analysis APIs. It proved that the upload-to-flag workflow could operate successfully before the addition of advanced machine learning models.

A.4 Phase 3: Visual Evidence Generation

This phase improved evidence quality through richer annotations, score displays, event-level galleries, and a dedicated results page. It significantly improved reviewer trust by providing clearer visual justification for flagged events.

A.5 Phase 4: Timeline Overlay and Clickable Markers

This phase added video streaming support, custom playback controls, clickable timeline markers, and severity-based filtering. These features made the dashboard substantially more efficient for real-world review tasks.

A.6 Phase 5: Advanced ML Detection Modules

This phase integrated YOLOv8 object detection and YOLOv8 pose estimation into the pipeline. As a result, the system moved beyond generic motion cues and gained the ability to identify suspicious objects and body behaviors more meaningfully.

A.7 Phase 6: Model Integration and Feature Fusion

This phase refined multimodal fusion by combining overlapping detector outputs, improving classification priority rules, and generating more coherent event explanations. It strengthened the system's ability to convert low-level detections into reviewer-friendly event summaries.

A.8 Phase 7: Dashboard, Exports, and Reporting

This phase completed the reviewer workflow by adding export routes for JSON, CSV, ZIP-based evidence bundles, and print-friendly reporting support. At this point, MulTiCheat became a much more complete end-to-end academic review tool.

Phase	Date	Tests re-reported	Primary outcome
0	2026-04-15	Initial smoke tests	Project scaffolding, back-end/frontend setup, config and README
1	2026-04-15	30/30 passed	Video upload, validation, metadata extraction, frame sampling
2	2026-04-15	44/44 passed	Baseline motion analysis pipeline, event aggregation, evidence APIs
3	2026-04-15	57/57 passed	Rich visual evidence generation and results gallery workflow
4	2026-04-16	59/59 passed	Clickable timeline markers and HTTP range-based video streaming
5–7	2026-04-16	64+ tests re-reported	Advanced ML detection, feature fusion, dashboard exports and reporting

Table 6: Implementation phase summary derived from the Markdown progress log.

References

- [1] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [2] Z. Cao, G. Hidalgo Martinez, T. Simon, S.-E. Wei, and Y. Sheikh, “OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2021.
- [3] C. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
- [4] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proceedings of the European Conference on Computer Vision*, 2022.
- [5] G. Jocher *et al.*, “Ultralytics YOLOv8,” GitHub repository, 2023.
- [6] N. Ruiz, E. Chong, and J. M. Rehg, “Fine-Grained Head Pose Estimation Without Key-points,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2018, pp. 2074–2083.
- [7] K. A. F. Mora and J.-M. Odobez, “Geometric Generative Gaze Estimation (G3E) for Remote RGB-D Cameras,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 1773–1780.
- [8] S. Nonaka, S. Nobuhara, and K. Nishino, “Dynamic 3D Gaze From Afar: Deep Gaze Estimation From Temporal Eye-Head-Body Coordination,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 2192–2201.
- [9] T.-Y. Yang, Y.-T. Chen, Y.-Y. Lin, and Y.-Y. Chuang, “FSA-Net: Learning Fine-Grained Structure Aggregation for Head Pose Estimation From a Single Image,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1087–1096.